

Linguagem SQL

DDL, DML e DQL — seguindo o Roteiro Prático

Unidade 5 · Banco de Dados · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Nesta aula

Esta aula **explica a sintaxe da SQL**, organizada como o **Roteiro Prático** — depois você **pratica no próprio roteiro**:

- **Parte 1 — DDL**: montar e evoluir a estrutura (tabelas, colunas, índices, views, chaves).
- **Parte 2 — DML**: inserir, atualizar e apagar dados.
- **Parte 3 — DQL**: consultar com `SELECT`, em **6 fases** que sobem de nível.

 Foco na **sintaxe** (Oracle); os **exercícios** você faz no **Roteiro Prático de SQL**, na **VM LabDatabase**. Ao final, a tabela de portabilidade entre SGBDs.

O domínio do roteiro

Partes 1 e 2 — Acadêmico

- ALUNOS , PROFESSORES , DISCIPLINAS
- OFERTAS (disciplina + professor + horário)
- ALUNOS_OFERTAS (matrícula — M:N)
- TELEFONES_ALUNOS (1:N)

Parte 3 — Vendas

- CLIENTES , PRODUTOS , UNIDADE_MEDIDA , UF
- PEDIDOS , ITENS_PEDIDOS

💡 O roteiro troca de domínio na Parte 3 (vendas) para praticar **consultas mais ricas** — mais junções e subconsultas.

Os quatro grupos da SQL

Grupo	Para quê	Comandos
DDL — <i>Definition</i>	estrutura	CREATE , ALTER , DROP
DML — <i>Manipulation</i>	dados	INSERT , UPDATE , DELETE
DQL — <i>Query</i>	consulta	SELECT
DCL — <i>Control</i>	permissões	GRANT , REVOKE

(há ainda o **TCL** — `COMMIT` , `ROLLBACK` , `SAVEPOINT` — visto na Unidade 6.)

Parte 1 · DDL

Montar e evoluir a estrutura

Passo 0 — deixar o script reexecutável

O roteiro **abre apagando tudo** (na ordem inversa das dependências), para você poder **rodar de novo** sem erro:

```
-- 1) tira as FKs    2) tira índices    3) tira tabelas    4) views e sequences
ALTER TABLE ALUNOS_OFERTAS DROP CONSTRAINT ALUNOS_OFERTA_FK;
DROP INDEX ALUNOS_NOME_IDX;
DROP TABLE ALUNOS_OFERTAS;
DROP VIEW ALUNOS_MATRICULADOS;
DROP SEQUENCE DISCIPLINAS_SEQ;
```

⚠ A ordem importa: não dá para apagar `ALUNOS` enquanto houver uma FK apontando para ela. Primeiro as **constraints**, depois as **tabelas**.

Passo 1 — CREATE TABLE (tabelas "cruas")

Primeiro as tabelas com as colunas essenciais — **sem chaves ainda**:

```
CREATE TABLE ALUNOS (  
    MATRICULA      NUMERIC      NOT NULL,  
    NOME           VARCHAR2(100) NOT NULL,  
    DATA_NASCIMENTO DATE       NOT NULL  
);  
  
CREATE TABLE DISCIPLINAS (  
    CODIGO_DISCIPLINA NUMERIC      NOT NULL,  
    NOME_DISCIPLINA   VARCHAR2(100) NOT NULL,  
    CARGA_HORARIA     NUMERIC(3)   NOT NULL,  
    EMENTA            VARCHAR2(4000) NOT NULL,  
    CODIGO_DISCIPLINA_DEPENDENCIA NUMERIC      -- auto-relacionamento  
);
```

💡 NOT NULL é uma **restrição**: o campo é obrigatório. As **chaves** (PK/FK) o roteiro adiciona **no fim**, com ALTER .

Tipos de dados mais comuns

Categoria	Oracle	PostgreSQL / MySQL
Texto	<code>VARCHAR2(n)</code>	<code>VARCHAR(n)</code>
Inteiro	<code>NUMERIC</code> / <code>NUMBER</code>	<code>INTEGER</code> / <code>INT</code>
Decimal	<code>NUMBER(p,s)</code>	<code>NUMERIC(p,s)</code>
Data	<code>DATE</code>	<code>DATE</code>
Data + hora	<code>TIMESTAMP</code>	<code>TIMESTAMP</code>

💡 Escolha o tipo pelo **significado** do dado — não guarde data como texto nem dinheiro como `float`.

Passo 2 — ALTER TABLE: evoluir as colunas

Raramente a tabela nasce perfeita. O roteiro usa o **ALTER** para **adicionar, modificar e renomear**:

```
-- adicionar / modificar
ALTER TABLE ALUNOS ADD EMAIL VARCHAR2(200);
ALTER TABLE ALUNOS MODIFY EMAIL VARCHAR2(250);
ALTER TABLE OFERTAS MODIFY DATA_CRIACAO DATE DEFAULT SYSDATE NOT NULL;

-- renomear tabela e colunas (padronizar os nomes)
ALTER TABLE TELEFONES RENAME TO TELEFONES_ALUNOS;
ALTER TABLE ALUNOS RENAME COLUMN MATRICULA TO MATRICULA_ALUNO;
ALTER TABLE ALUNOS RENAME COLUMN NOME TO NOME_ALUNO;
```

⚠ Mais adiante o roteiro faz `ALTER TABLE ALUNOS DROP COLUMN EMAIL;` — o `DROP COLUMN` apaga os dados da coluna e é irreversível sem backup.

Passo 3 — Sequences e Índices

```
-- sequência: gerar códigos automáticos (Oracle)
CREATE SEQUENCE DISCIPLINAS_SEQ;
-- ... depois: INSERT INTO DISCIPLINAS VALUES (DISCIPLINAS_SEQ.NEXTVAL, ...);

-- índices: acelerar buscas frequentes por nome/dia
CREATE INDEX ALUNOS_NOME_IDX      ON ALUNOS (NOME_ALUNO);
CREATE INDEX OFERTAS_DIA_IDX     ON OFERTAS (DIA_SEMANA);
CREATE INDEX DISCIPLINA_NOME_IDX  ON DISCIPLINAS (NOME_DISCIPLINA);
```

💡 Em PostgreSQL/MySQL o autoincremento vem de `SERIAL` / `AUTO_INCREMENT`, em vez de *sequence* explícita.

Passo 4 — Views: consultas com nome

```
CREATE VIEW ALUNOS_MATRICULADOS AS
  SELECT A.MATRICULA_ALUNO, A.NOME_ALUNO AS ALUNO, AO.SEMESTRE,
         O.DIA_SEMANA, P.NOME_PROFESSOR AS PROFESSOR, D.NOME_DISCIPLINA
  FROM ALUNOS_OFERTAS AO
  INNER JOIN ALUNOS      A ON AO.MATRICULA_ALUNO = A.MATRICULA_ALUNO
  INNER JOIN OFERTAS     O ON AO.CODIGO_OFERTA   = O.CODIGO_OFERTA
  INNER JOIN PROFESSORES P ON O.MATRICULA_PROFESSOR = P.MATRICULA_PROFESSOR
  INNER JOIN DISCIPLINAS D ON O.CODIGO_DISCIPLINA = D.CODIGO_DISCIPLINA;
```

Depois é só `SELECT * FROM ALUNOS_MATRICULADOS;` — a complexidade fica **escondida** atrás do nome.

Passo 5 — Chaves por último: PK e FK

Com as tabelas prontas, o roteiro **fecha a integridade**:

```
-- chaves primárias (simples e composta)
ALTER TABLE ALUNOS          ADD PRIMARY KEY (MATRICULA_ALUNO);
ALTER TABLE ALUNOS_OFERTAS ADD PRIMARY KEY (MATRICULA_ALUNO, CODIGO_OFERTA);

-- chaves estrangeiras (constraint nomeada)
ALTER TABLE OFERTAS
  ADD CONSTRAINT OFERTAS_PROFESSOR_FK
    FOREIGN KEY (MATRICULA_PROFESSOR)
    REFERENCES PROFESSORES (MATRICULA_PROFESSOR);
```

💡 **Por que no fim?** Uma FK só pode ser criada quando a tabela referenciada **e sua PK** já existem. Por isso a ordem do roteiro: tabelas → PKs → FKs.

Parte 2 · DML

Inserir, atualizar e apagar

INSERT

```
-- forma posicional
INSERT INTO ALUNOS
VALUES (40001, 'JOÃO GABRIEL', TO_DATE('02/04/1985', 'DD/MM/YYYY'));

-- forma explícita (recomendada): lista as colunas
INSERT INTO ALUNOS (MATRICULA_ALUNO, NOME_ALUNO, DATA_NASCIMENTO)
VALUES (40002, 'JOÃO JOSÉ', TO_DATE('31/12/2001', 'DD/MM/YYYY'));
```

💡 Listar as colunas deixa o comando **imune** a mudanças na ordem/quantidade de colunas da tabela.

UPDATE

```
-- SEMPRE com WHERE! (senão altera a tabela inteira)
UPDATE ALUNOS
  SET DATA_NASCIMENTO = TO_DATE('29/04/2011','DD/MM/YYYY')
  WHERE MATRICULA_ALUNO = 40004;

-- atualização por conjunto
UPDATE PROFESSORES
  SET FORMACAO = 'PHD'
  WHERE FORMACAO = 'DOUTORADO';
```

⚠ Um `UPDATE` sem `WHERE` altera **todas** as linhas. Confira o filtro com um `SELECT` antes.

DELETE

```
DELETE FROM DISCIPLINAS
WHERE CODIGO_DISCIPLINA IN (85853, 75189);

-- apagar professores que não têm nenhuma oferta
DELETE FROM PROFESSORES P
WHERE NOT EXISTS (SELECT 1 FROM OFERTAS O
                  WHERE O.MATRICULA_PROFESSOR = P.MATRICULA_PROFESSOR);
```

💡 Um DELETE só vira **definitivo** ao confirmar a transação (COMMIT) — desfazer/confirmar é assunto da **Unidade 6**.

Parte 3 · DQL

SELECT em 6 fases (domínio de vendas)

Fase 1 — projeção, DISTINCT e ORDER BY

```
-- todas as colunas x só as que interessam
SELECT PRO.NOME_PRODUTO FROM PRODUTOS PRO;

-- valores únicos, ordenados
SELECT DISTINCT CLI.CIDADE, CLI.UF, CLI.CEP
FROM CLIENTES CLI
ORDER BY CLI.UF;
```

- **Projeção**: escolher **colunas** (em vez de `SELECT *`).
- **DISTINCT** remove linhas repetidas; **ORDER BY** ordena (`ASC` padrão, `DESC` inverte).

Fase 1 — filtros: comparação, faixa, lista, texto

```
SELECT PED.* FROM PEDIDOS PED
WHERE PED.CODIGO_CLIENTE = 4 AND PED.VALOR_LIQUIDO > 10000;

SELECT PED.* FROM PEDIDOS PED
WHERE PED.VALOR_TOTAL NOT BETWEEN 100 AND 5000;           -- faixa

SELECT CLI.* FROM CLIENTES CLI WHERE CLI.UF IN ('ES','MG'); -- lista

SELECT PRO.* FROM PRODUTOS PRO
WHERE UPPER(PRO.NOME_PRODUTO) LIKE 'MA_____';          -- MA + 6 chars
```

💡 No LIKE: % = qualquer sequência · _ = um caractere. Use ... LIKE '%A_P%' ESCAPE '\ ' para procurar o _ literal. E teste ausência com IS NULL (nunca = NULL).

Fase 1 — outro exemplo (OR e NOT IN)

```
-- fora de uma faixa de códigos
SELECT CLI.* FROM CLIENTES CLI
WHERE CLI.CODIGO_CLIENTE < 5 OR CLI.CODIGO_CLIENTE > 25;

-- excluir alguns estados
SELECT CLI.* FROM CLIENTES CLI
WHERE CLI.UF NOT IN ('RJ', 'SP');
```

💡 OR combina condições (basta uma ser verdadeira); NOT IN nega a lista.



Pratique a Fase 1 no Roteiro Prático de SQL (Parte 3).

Fase 2 — junções (INNER JOIN)

```
SELECT PED.CODIGO_PEDIDO, CLI.NOME_CLIENTE,  
       PRO.NOME_PRODUTO, UE.DESCRICAO_UNIDADE_MEDIDA  
FROM PEDIDOS PED  
INNER JOIN CLIENTES      CLI ON PED.CODIGO_CLIENTE = CLI.CODIGO_CLIENTE  
INNER JOIN ITENS_PEDIDOS ITE ON PED.CODIGO_PEDIDO  = ITE.CODIGO_PEDIDO  
INNER JOIN PRODUTOS      PRO ON ITE.CODIGO_PRODUTO = PRO.CODIGO_PRODUTO  
INNER JOIN UNIDADE_MEDIDA UE ON PRO.CODIGO_UNIDADE_MEDIDA = UE.CODIGO_UNIDADE_MEDIDA;
```

💡 Cada JOIN ... ON costura duas tabelas pela chave. Dá para renomear a saída com alias (AS "PREÇO DO PRODUTO") e até criar **colunas calculadas** (PRECO_PRODUTO * 1.3).

Fase 2 — outro exemplo (alias, coluna calculada, subconsulta)

```
SELECT PRO.NOME_PRODUTO AS "NOME DO PRODUTO",  
       PRO.PRECO_PRODUTO AS "PREÇO ATUAL",  
       (PRO.PRECO_PRODUTO * 1.3) AS "PREÇO PROJETADO"  
FROM PRODUTOS PRO  
WHERE PRO.CODIGO_PRODUTO IN (SELECT ITE.CODIGO_PRODUTO FROM ITENS_PEDIDOS ITE);
```

💡 Alias entre aspas aceita **acentos e espaços**; dá para calcular colunas na hora (* 1.3) e filtrar por uma **subconsulta** (só produtos que já foram pedidos).



Pratique a Fase 2 no Roteiro Prático de SQL (Parte 3).

Fase 3 — agregação, GROUP BY e HAVING

```
SELECT MIN(V valor_total) AS MINIMO, MAX(V valor_total) AS MAXIMO,  
       SUM(V valor_total) AS TOTAL, ROUND(AVG(V valor_total),2) AS MEDIA,  
       COUNT(1) AS QTDE  
FROM PEDIDOS;  
  
SELECT CLI.CODIGO_CLIENTE, ROUND(AVG(PED.V valor_total),2) AS MEDIA_POR_CLIENTE  
FROM CLIENTES CLI  
  INNER JOIN PEDIDOS PED ON CLI.CODIGO_CLIENTE = PED.CODIGO_CLIENTE  
GROUP BY CLI.CODIGO_CLIENTE  
HAVING ROUND(AVG(PED.V valor_total),2) > 8000;
```

- **GROUP BY** agrupa antes de agregar; **HAVING** filtra **grupos** (o **WHERE** filtra **linhas**, antes).

Fase 3 — LEFT JOIN e subconsulta escalar

```
-- produtos que NUNCA foram pedidos (o que existe de um lado e não do outro)
SELECT PRO.*
  FROM PRODUTOS PRO
  LEFT OUTER JOIN ITENS_PEDIDOS ITE ON PRO.CODIGO_PRODUTO = ITE.CODIGO_PRODUTO
 WHERE ITE.CODIGO_PRODUTO IS NULL;

-- produtos acima da média geral de preço (subconsulta escalar)
SELECT PRO.NOME_PRODUTO, PRO.PRECO_PRODUTO
  FROM PRODUTOS PRO
 WHERE PRO.PRECO_PRODUTO > (SELECT AVG(PRECO_PRODUTO) FROM PRODUTOS);
```

Fase 3 — outro exemplo (COUNT por grupo)

```
-- quantos clientes (que já pediram) cada UF tem
SELECT CLI.UF, COUNT(CLI.CODIGO_CLIENTE) AS QUANTIDADE_CLIENTES
  FROM CLIENTES CLI
 WHERE CLI.CODIGO_CLIENTE IN (SELECT PED.CODIGO_CLIENTE FROM PEDIDOS PED)
 GROUP BY CLI.UF
 ORDER BY QUANTIDADE_CLIENTES DESC;
```

💡 O WHERE filtra as **linhas** antes; o GROUP BY forma os grupos; a agregação (COUNT) resume cada grupo; o ORDER BY ordena pelo resultado.



Pratique a Fase 3 no Roteiro Prático de SQL (Parte 3).

Fase 4 — subconsultas de conjunto (IN / NOT IN)

```
-- clientes que pediram no dia 13 MAS não no dia 02
SELECT CLI.*
  FROM CLIENTES CLI
 WHERE CLI.CODIGO_CLIENTE IN      (SELECT PED.CODIGO_CLIENTE FROM PEDIDOS PED
                                   WHERE PED.DATA_PEDIDO = TO_DATE('13/01/2007','DD/MM/YYYY'))
    AND CLI.CODIGO_CLIENTE NOT IN (SELECT PED.CODIGO_CLIENTE FROM PEDIDOS PED
                                   WHERE PED.DATA_PEDIDO = TO_DATE('02/01/2007','DD/MM/YYYY'));
```

💡 IN + NOT IN juntos expressam **diferença de conjuntos** ("está em A e não em B"). A subconsulta pode ainda ter seus próprios JOIN s.

Fase 4 — outro exemplo (subconsulta com junções)

```
-- produtos NUNCA pedidos por clientes do RJ
SELECT PRO.* FROM PRODUTOS PRO
WHERE PRO.CODIGO_PRODUTO NOT IN (
    SELECT ITE.CODIGO_PRODUTO
    FROM ITENS_PEDIDOS ITE
    INNER JOIN PEDIDOS PED ON ITE.CODIGO_PEDIDO = PED.CODIGO_PEDIDO
    INNER JOIN CLIENTES CLI ON CLI.CODIGO_CLIENTE = PED.CODIGO_CLIENTE
    WHERE CLI.UF = 'RJ');
```

💡 A subconsulta pode ter os seus próprios JOIN s e WHERE — ela monta a lista de "produtos pedidos por clientes do RJ", e o NOT IN pega o **complemento**.

💻 **Pratique a Fase 4** no Roteiro Prático de SQL (Parte 3).

Fase 5 — outer joins encadeados e NULLS

```
-- total por produto, incluindo os que não venderam (total NULL primeiro)
SELECT PRO.CODIGO_PRODUTO, PRO.NOME_PRODUTO, PRO.PRECO_PRODUTO,
       SUM(PED.VALOR_TOTAL) AS TOTAL_POR_PRODUTO
FROM PRODUTOS PRO
LEFT JOIN ITENS_PEDIDOS ITE ON PRO.CODIGO_PRODUTO = ITE.CODIGO_PRODUTO
LEFT JOIN PEDIDOS      PED ON ITE.CODIGO_PEDIDO = PED.CODIGO_PEDIDO
GROUP BY PRO.CODIGO_PRODUTO, PRO.NOME_PRODUTO, PRO.PRECO_PRODUTO
ORDER BY TOTAL_POR_PRODUTO NULLS FIRST;
```

💡 Encadear LEFT JOIN preserva os produtos mesmo sem item/pedido; NULLS FIRST / NULLS LAST controlam onde os nulos aparecem na ordenação.

Fase 5 — outro exemplo (LEFT JOIN encadeado)

```
-- todos os clientes e, se houver, os produtos que compraram
SELECT CLI.NOME_CLIENTE, PRO.NOME_PRODUTO
FROM CLIENTES CLI
LEFT JOIN PEDIDOS      PED ON CLI.CODIGO_CLIENTE = PED.CODIGO_CLIENTE
LEFT JOIN ITENS_PEDIDOS ITE ON PED.CODIGO_PEDIDO = ITE.CODIGO_PEDIDO
LEFT JOIN PRODUTOS     PRO ON ITE.CODIGO_PRODUTO = PRO.CODIGO_PRODUTO
ORDER BY CLI.NOME_CLIENTE;
```

💡 A **cadeia** de LEFT JOIN preserva o cliente em cada passo — quem não comprou aparece com NULL no produto, em vez de sumir do resultado.



Pratique a Fase 5 no Roteiro Prático de SQL (Parte 3).

Fase 6 — subconsulta no **SELECT** / **HAVING** e views

```
-- comparar o total de um produto com um valor calculado por subconsulta
SELECT PRO.NOME_PRODUTO, SUM(PED.VALOR_TOTAL) AS TOTAL_VENDIDO
  FROM PRODUTOS PRO
  INNER JOIN ITENS_PEDIDOS ITE ON ITE.CODIGO_PRODUTO = PRO.CODIGO_PRODUTO
  INNER JOIN PEDIDOS      PED ON ITE.CODIGO_PEDIDO  = PED.CODIGO_PEDIDO
 GROUP BY PRO.NOME_PRODUTO
HAVING SUM(PED.VALOR_TOTAL) > (SELECT SUM(PED.VALOR_TOTAL) FROM ...); -- ex.: total do 'SAL'

-- guardar uma consulta pronta como VIEW
CREATE OR REPLACE VIEW PRODUTOS_MEDIDAS AS
  SELECT PRO.NOME_PRODUTO, UE.DESCRICAO_UNIDADE_MEDIDA
  FROM PRODUTOS PRO
  INNER JOIN UNIDADE_MEDIDA UE ON PRO.CODIGO_UNIDADE_MEDIDA = UE.CODIGO_UNIDADE_MEDIDA;
```

💡 O fecho do roteiro: consultas que usam o resultado de **outra consulta** — e transformam as boas em **views** reutilizáveis.

Fase 6 — outro exemplo (VIEW a partir de agregação)

```
CREATE OR REPLACE VIEW TOTAL_CLIENTE_VITORIA AS
  SELECT CLI.NOME_CLIENTE, SUM(PED.VALOR_TOTAL) AS TOTAL_POR_CLIENTE
  FROM CLIENTES CLI
  INNER JOIN PEDIDOS PED ON CLI.CODIGO_CLIENTE = PED.CODIGO_CLIENTE
  WHERE UPPER(CLI.CIDADE) = 'VITORIA'
  GROUP BY CLI.NOME_CLIENTE
  HAVING SUM(PED.VALOR_TOTAL) > (SELECT AVG(PED.VALOR_TOTAL) FROM ...);
```

💡 Uma **view** pode encapsular uma consulta inteira — com `GROUP BY`, `HAVING` e até subconsulta. Depois, é só `SELECT * FROM TOTAL_CLIENTE_VITORIA;`.



Pratique a Fase 6 no Roteiro Prático de SQL (Parte 3).

Portabilidade entre SGBDs

Recurso	Oracle	PostgreSQL	MySQL
Texto	<code>VARCHAR2</code>	<code>VARCHAR</code>	<code>VARCHAR</code>
Data literal	<code>TO_DATE('..','DD/MM/YYYY')</code>	<code>DATE '2026-01-01'</code>	<code>'2026-01-01'</code>
Data/hora atual	<code>SYSDATE</code>	<code>NOW()</code>	<code>NOW()</code>
Autoincremento	<code>SEQUENCE</code>	<code>SERIAL</code>	<code>AUTO_INCREMENT</code>
Trata <code>NULL</code>	<code>NVL(x,y)</code>	<code>COALESCE(x,y)</code>	<code>IFNULL / COALESCE</code>
Limitar linhas	<code>FETCH FIRST n ROWS</code>	<code>LIMIT n</code>	<code>LIMIT n</code>

Para praticar — o Roteiro Prático de SQL

Você executa tudo na VM, na mesma ordem desta aula:

- **Parte 1 — DDL:** montar o esquema acadêmico (create → alter → índices → views → PK/FK).
- **Parte 2 — DML:** insert/update/delete e consultas básicas.
- **Parte 3 — DQL:** as **6 fases** de `SELECT` (vendas), da projeção às subconsultas e views.

 Esta aula explica a **sintaxe**; os **exercícios** estão no **Roteiro Prático de SQL (PDF)** — com os **scripts .sql** por parte e os **dados para inserir** (também no **Sumário → Unidade 5**). Trabalhado **em aula**, na **VM LabDatabase**.

Bibliografia

- ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7ª ed. Pearson, 2019.
- DATE, C. J. **Introdução a Sistemas de Banco de Dados**. 8ª ed. Elsevier, 2004.
- SILBERSCHATZ, A.; KORTH, H.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7ª ed. Elsevier, 2020.

Dúvidas?

howard.cruz@faesa.br

Próximo →
Procedimentos Armazenados